ГУАП
КАФЕДРА № 41

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ

ПРЕПОДАВАТЕЛЬ

ассистент		А.С. Раскопина
должность, уч. степень, звание	подпись, дата	инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №3

РЕКУРРЕНТНЫЕ НЕЙРОННЫЕ СЕТИ

по курсу:

МАШИННОЕ ОБУЧЕНИЕ И БОЛЬШИЕ ДАННЫЕ

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. №	4316		М.С. Пронь
		подпись, дата	инициалы, фамилия

Санкт-Петербург 2026

СОДЕРЖАНИЕ

1 Цель работы 3

2 Описание датасета 3

3 Описание обученных моделей и их адаптация 5

4 Процесс обучения 7

5 Сравнение моделей, анализ результатов 9

6 Выводы 10

1 Цель работы

Изучить принципы работы рекуррентных нейронных сетей для прогнозирования временных рядов.

2 Описание датасета

В работе был использован датасет Traffic Time Series Dataset для 20 варианта, содержащий почасовые наблюдения дорожного трафика, погодных условий и событий. Была выполнена загрузка данных из открытого источника Kaggle и была проверена структура таблицы (рис. 1).

```
csv_files = [f for f in os.listdir(path) if f.endswith('.csv')]

df = pd.read_csv(os.path.join(path, csv_files[0]))
df.head()
```

	Timestamp	Weather	Events	Traffic Volume
0	2023-01-01 00:00:00	Cloudy	False	803.0
1	2023-01-01 01:00:00	Clear	False	861.0
2	2023-01-01 02:00:00	Clear	False	582.0
3	2023-01-01 03:00:00	Cloudy	False	236.0
4	2023-01-01 04:00:00	Cloudy	False	569.0

Рисунок 1 – Исходный датасет

Был выведен первичный анализ данных: информация о типах столбцов, статистические характеристики, наличие пропусков и базовые свойства признаков (рис. 2). На данном этапе было установлено, что данные пригодны для построения моделей после стандартной предобработки.

```python
print(df.info())
print('\nСтатистика')
print(df.describe())
print('\n',df.isnull().sum())
print(f'\nВсего пропусков: {df.isnull().sum().sum()}')
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8736 entries, 0 to 8735
Data columns (total 4 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   Timestamp       8736 non-null   object
 1   Weather         8736 non-null   object
 2   Events          8736 non-null   bool
 3   Traffic Volume  8736 non-null   float64
dtypes: bool(1), float64(1), object(2)
memory usage: 213.4+ KB
None

Статистика
       Traffic Volume
count     8736.000000
mean      1281.395833
std        727.291462
min        118.000000
25%        893.000000
50%       1123.000000
75%       1415.000000
max       6465.000000

 Timestamp         0
Weather           0
Events            0
Traffic Volume    0
dtype: int64

Всего пропусков: 0
```

Рисунок 2 – Анализ датасета

Был построен и проанализирован график временного ряда для оценки динамики и возможной сезонности (рис. 3). Было показано, что ряд обладает выраженной изменчивостью, что обосновывает применение рекуррентных архитектур.

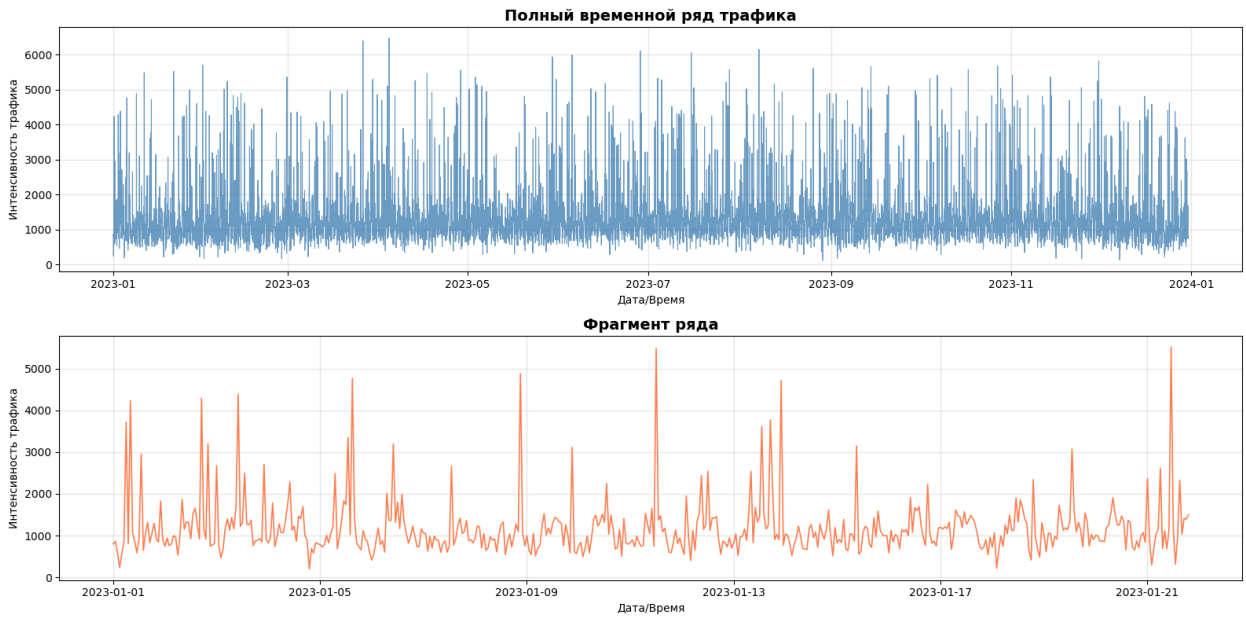

Рисунок 3 – Временные ряды трафика

3 Описание обученных моделей и их адаптация

В рамках лабораторной работы были реализованы и исследованы три архитектуры рекуррентных сетей: классическая RNN, LSTM и GRU. Для всех моделей была использована единая схема адаптации: одинаковая длина входного окна, сопоставимое число скрытых слоев, одинаковый размер скрытого состояния и единый выходной блок регрессии. Такой подход позволил выполнить корректное сравнение моделей в одинаковых условиях.

Ниже приведен листинг реализации моделей, использованный в работе.

Листинг 1 – Листинг реализации RNN, LSTM и GRU

```
class RNNModel(nn.Module):
    def __init__(self, input_size=1, hidden_size=64, num_layers=2,
dropout=0.2):
        super().__init__()
        self.rnn = nn.RNN(
            input_size=input_size,
            hidden_size=hidden_size,
            num_layers=num_layers,
            dropout=dropout if num_layers > 1 else 0.0,
            batch_first=True
        )
        self.fc = nn.Sequential(
            nn.Linear(hidden_size, 32),
            nn.ReLU(),
            nn.Dropout(dropout),
            nn.Linear(32, 1)
        )
```

```
class LSTMModel(nn.Module):
    """LSTM - Long Short-Term Memory"""
    def __init__(self, input_size=1, hidden_size=64, num_layers=2,
dropout=0.2):
        super().__init__()
        self.lstm = nn.LSTM(
            input_size=input_size,
            hidden_size=hidden_size,
            num_layers=num_layers,
            dropout=dropout if num_layers > 1 else 0.0,
            batch_first=True
        )
        self.fc = nn.Sequential(
            nn.Linear(hidden_size, 32),
            nn.ReLU(),
            nn.Dropout(dropout),
            nn.Linear(32, 1)
        )


class GRUModel(nn.Module):
    """GRU - Gated Recurrent Unit"""
    def __init__(self, input_size=1, hidden_size=64, num_layers=2,
dropout=0.2):
        super().__init__()
        self.gru = nn.GRU(
            input_size=input_size,
            hidden_size=hidden_size,
            num_layers=num_layers,
            dropout=dropout if num_layers > 1 else 0.0,
            batch_first=True
        )
        self.fc = nn.Sequential(
            nn.Linear(hidden_size, 32),
            nn.ReLU(),
            nn.Dropout(dropout),
            nn.Linear(32, 1)
        )


for ModelClass, name in [(RNNModel, 'RNN'), (LSTMModel, 'LSTM'),
(GRUModel, 'GRU')]:
    model = ModelClass().to(DEVICE)
    n_params = sum(p.numel() for p in model.parameters() if
p.requires_grad)
    print(f'{name}: {n_params:,} обучаемых параметров')
    del model
```

Была выполнена предварительная инженерия признаков и формирование многомерного входа модели: к целевому признаку трафика были добавлены погодные и календарные признаки. Было выполнено масштабирование признаков и было сформировано скользящее окно фиксированной длины (рис. 5).

```python
FEATURE_COLS = ['Traffic Volume', 'Weather_enc', 'Events_enc', 'hour', 'dayofweek', 'is_weekend']
TARGET_COL   = 'Traffic Volume'

data = df[FEATURE_COLS].values.astype(np.float32)

n = len(data)
train_end = int(n * 0.70)
val_end   = int(n * 0.85)

train_raw = data[:train_end]
val_raw   = data[train_end:val_end]
test_raw  = data[val_end:]

scaler_X = MinMaxScaler()
train_scaled = scaler_X.fit_transform(train_raw)
val_scaled   = scaler_X.transform(val_raw)
test_scaled  = scaler_X.transform(test_raw)

scaler_y = MinMaxScaler()
scaler_y.fit(train_raw[:, 0].reshape(-1, 1))

print(f"Train: {len(train_scaled)}, Val: {len(val_scaled)}, Test: {len(test_scaled)}")
print(f"Признаков: {train_scaled.shape[1]}")
```

```
Train: 6115, Val: 1310, Test: 1311
Признаков: 6
```

Рисунок 4 – Предобработка данных

4 Процесс обучения

Было выполнено обучение трех моделей на одинаковых обучающей, валидационной и тестовой выборках. В процессе обучения была использована оптимизация Adam, функция потерь MSE и механизм ранней остановки по валидационной ошибке. Такой протокол позволил снизить риск переобучения и обеспечить сопоставимость результатов.

Был выведен сводный результат этапа обучения моделей (рис. 6), после чего была выполнена визуализация кривых обучения для анализа сходимости (рис. 7).

```
Обучение модели: RNN
Epoch   1/25 | Train Loss: 0.013023 | Val Loss: 0.012196
Epoch   5/25 | Train Loss: 0.012223 | Val Loss: 0.011912
Epoch  10/25 | Train Loss: 0.012077 | Val Loss: 0.011757
Epoch  15/25 | Train Loss: 0.012112 | Val Loss: 0.011824
Epoch  20/25 | Train Loss: 0.011898 | Val Loss: 0.011782
Ранняя остановка на эпохе 21

Результаты RNN:
  MSE: 481850.2812
  MAE: 405.3154
  R²:  0.0489
Обучение модели: LSTM
Epoch   1/25 | Train Loss: 0.016117 | Val Loss: 0.012957
Epoch   5/25 | Train Loss: 0.012831 | Val Loss: 0.011744
Epoch  10/25 | Train Loss: 0.012361 | Val Loss: 0.011756
Epoch  15/25 | Train Loss: 0.012175 | Val Loss: 0.011824
Epoch  20/25 | Train Loss: 0.012131 | Val Loss: 0.011670
Epoch  25/25 | Train Loss: 0.012000 | Val Loss: 0.011732

Результаты LSTM:
  MSE: 481867.5000
  MAE: 404.1168
  R²:  0.0489
Обучение модели: GRU
Epoch   1/25 | Train Loss: 0.017534 | Val Loss: 0.013506
Epoch   5/25 | Train Loss: 0.012773 | Val Loss: 0.011792
Epoch  10/25 | Train Loss: 0.012319 | Val Loss: 0.011799
Epoch  15/25 | Train Loss: 0.012167 | Val Loss: 0.011699
Epoch  20/25 | Train Loss: 0.012114 | Val Loss: 0.011693
Epoch  25/25 | Train Loss: 0.011968 | Val Loss: 0.011692

Результаты GRU:
  MSE: 479891.9375
  MAE: 401.3103
  R²:  0.0528
```

Рисунок 5 – Обучение моделей

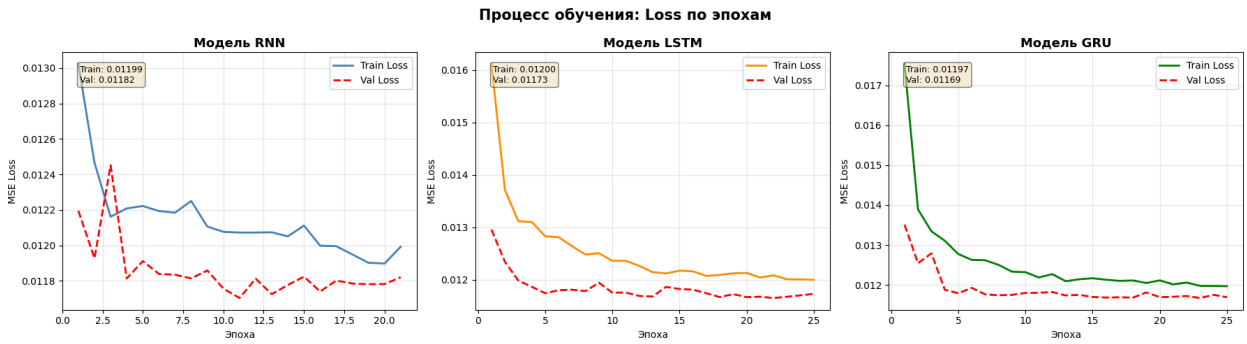

Рисунок 6 – Графики процесса обучения

5 Сравнение моделей, анализ результатов

Было выполнено сравнение качества моделей по метрикам MSE, MAE и R^2 (рис. 8). Было установлено, что модели LSTM и GRU демонстрируют более высокое качество прогнозирования по сравнению с классической RNN, что согласуется с их архитектурными особенностями и лучшей способностью учитывать долгосрочные зависимости.

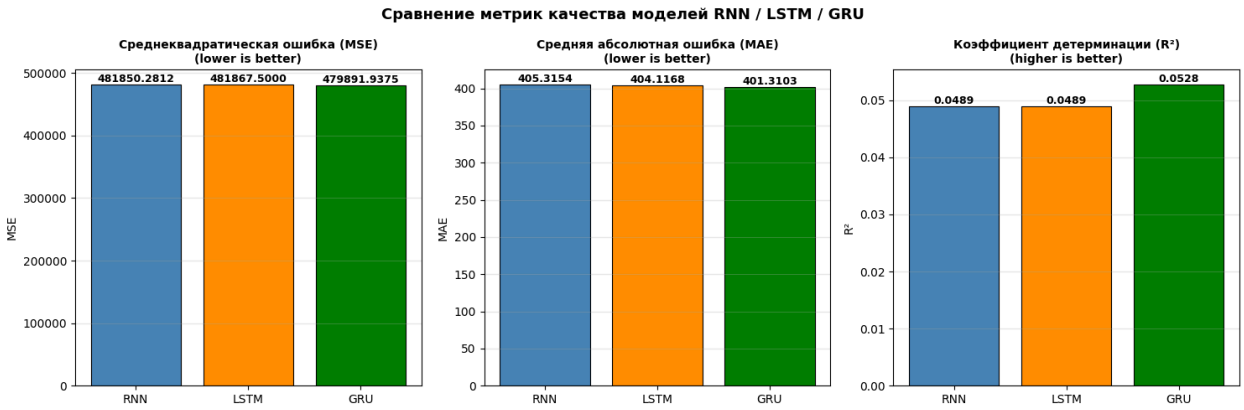

Рисунок 7 – Сравнение метрик качества

Дополнительно было выполнено сравнение предсказаний моделей с реальными значениями временного ряда (рис. 9). Было показано, что современные gated-архитектуры (LSTM, GRU) лучше повторяют динамику целевого сигнала и формируют более устойчивый прогноз.

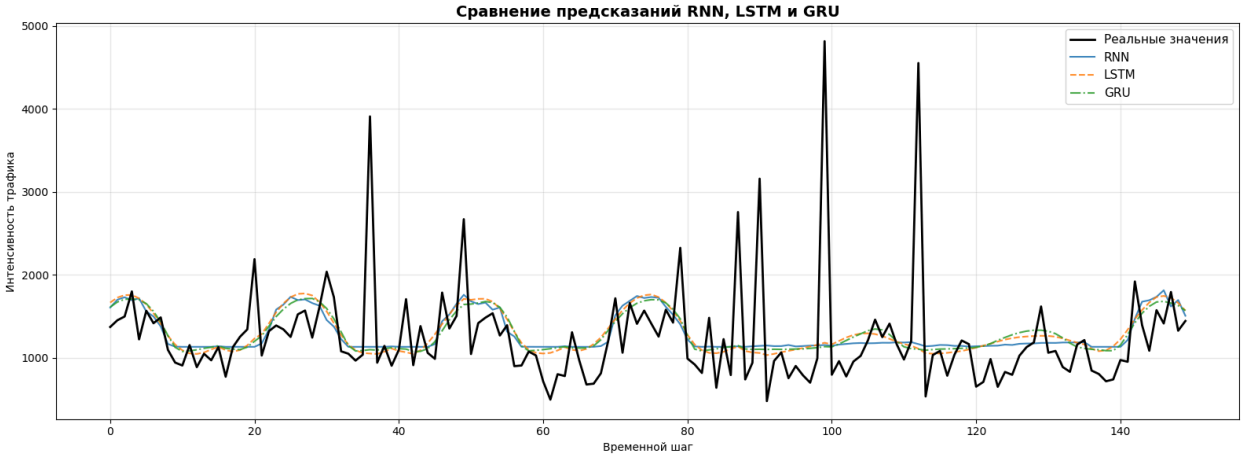

Рисунок 8 – Сравнение предсказаний моделей

6 Выводы

В ходе лабораторной работы была исследована задача прогнозирования временного ряда трафика с использованием рекуррентных нейронных сетей. Был выполнен полный цикл работ: загрузка и анализ датасета, предобработка признаков, обучение трех моделей (RNN, LSTM, GRU), оценка качества и интерпретация результатов.

По итогам сравнения было установлено, что модели LSTM и GRU обеспечивают более точный прогноз по метрикам MSE, MAE и R^2 по сравнению с базовой RNN. Полученные результаты соответствуют цели работы и подтверждают целесообразность применения gated-рекуррентных архитектур для задач прогнозирования трафика.

Код лабораторной работы приведен в ноутбуке: https://colab.research.google.com/drive/1QRQkVEfiyLFfpJ1WjRFh6HMPyUuZWAA7?usp=sharing